Mastermind Pod Automation System · Implementation Plan

Context

Dan Martell runs a coaching programme called **Elite** with ~1,000 active clients. Currently there is no automated system for connecting clients into peer groups. The goal is to build a fully autonomous, AI-powered system that:

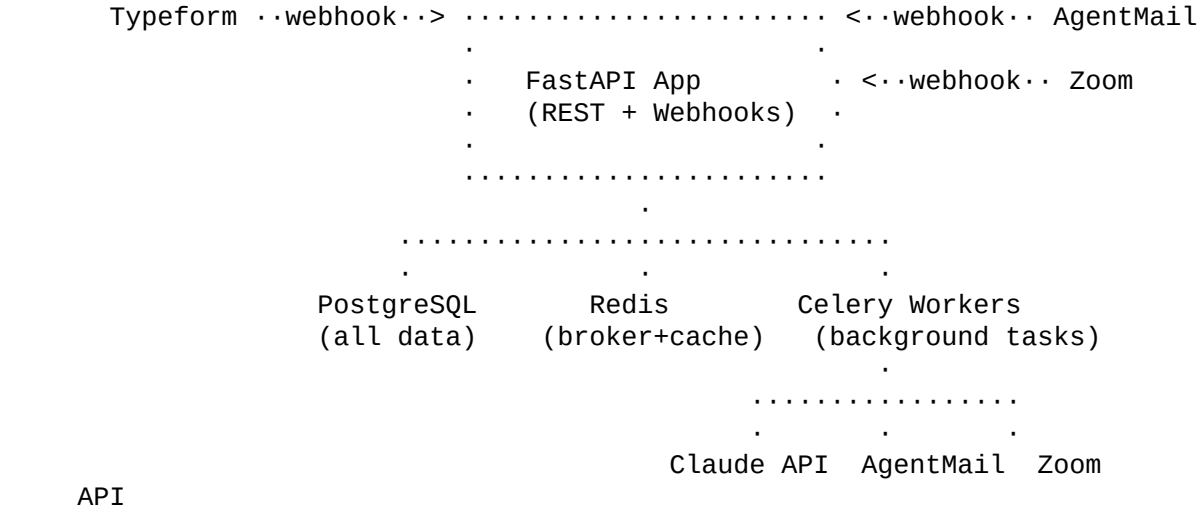
- Ingests client intake forms from **Typeform**
- Groups clients into **self-directed mastermind pods of up to 8** (same region/timezone, non-competitive industries)
- Handles all communication via **AgentMail** (invitation, confirmation, bio approval, reminders, follow-ups)
- Schedules and hosts **monthly Zoom calls** per pod, tracks attendance, retrieves transcripts
- Enforces a **2-consecutive-miss removal rule** and automatically backfills pods
- Publishes **dynamic pod member pages** at non-guessable URLs
- Provides an **admin dashboard** with full visibility into all operations

---

Tech Stack

| Concern | Choice | |---------|--------| | **Language** | Python 3.12+ | | **LLM** | Anthropic Claude API (Claude Opus 4.6) via `anthropic` SDK | | **Email** | AgentMail (`pip install agentmail`) · inboxes, send/receive, webhooks‖ **Intake Forms** | Typeform Responses API + Typeform Webhooks | | **Video** | Zoom API (Server-to-Server OAuth) · Zoom Business/Pro with cloud recording | | **Web Framework** | FastAPI + Uvicorn | | **Task Queue** | Celery 5.4+ with Redis broker | | **Scheduler** | Celery Beat | | **Database** | PostgreSQL 16 (asyncpg driver, SQLAlchemy 2.0 async ORM) | | **Cache/Broker** | Redis 7 | | **Admin UI** | FastAPI + Jinja2 + HTMX + Tailwind CSS (server-rendered, no JS framework) | | **Migrations** | Alembic | | **HTTP Client** | httpx (async, for Zoom/Typeform API calls) | | **Hosting** | Self-hosted VPS (4 vCPU, 8 GB RAM sufficient for 1,000 clients) | | **Reverse Proxy** | Caddy (automatic TLS) | | **Containerization** | Docker + Docker Compose |

Architecture Overview



Data flows:

1 Typeform webhook fires on new submission · FastAPI stores raw data · Celery task sends to Claude for

structured analysis
2. Daily batch: matching engine scores unassigned clients pairwise, groups into pods
3. AgentMail sends personalized invitations; webhook receives replies; Claude classifies yes/no/maybe
4. On enough confirmations: pod activates · bios generated · Zoom meeting created · confirmation emails sent
5. Monthly: reminders sent 24h and 1h before · meeting happens · attendance checked via Zoom Report API · transcript fetched · Claude extracts summary/action items · follow-up email sent
6. Miss tracking: 1st miss = warning email, 2nd consecutive miss = removal + backfill triggered

---

Database Schema (Key Tables)

`clients`

- `id` UUID PK, `external_id` (Typeform response ID), `email`, `first_name`, `last_name`
- `company_name`, `company_website`, `phone`
- `intake_form_raw` JSONB (full Typeform response)
- Claude-analyzed fields: `industry`, `industry_niche`, `business_model`, `annual_revenue_range`, `employee_count_range`, `business_stage`, `timezone` (IANA), `region`, `country`, `city`, `primary_challenge`, `goals`
- `analysis_confidence` FLOAT, `analysis_raw` JSONB
- `status`: `intake_received` ·`analyzed` ·`invited` ·`confirmed` ·`active` ·`removed`/`declined`/`churned`
- `bio_summary` TEXT, `bio_status`: `not_generated` ·`generated` · `sent_for_approval` ·`approved`
- `agentmail_inbox_id` (for threading)

`pods`

- `id` UUID PK, `name` (e.g., "Pod Alpha"), `slug`
- `access_token` VARCHAR(64) UNIQUE · 32 random bytes hex-encoded for non-guessable public URL
- `status`: `forming` ·`confirming` ·`active` ·`paused`/`disbanded`
- `max_members` INT (default 8), `target_timezone`, `target_region`
- `meeting_cadence` (default `monthly`), `preferred_day_of_week`, `preferred_time_utc`, `meeting_duration_minutes` (default 60)
- `zoom_meeting_id`, `zoom_join_url`
- `formation_score` FLOAT

`pod_memberships`

- `pod_id` FK, `client_id` FK (unique together)
- `status`: `invited` ·`confirmed` ·`active` ·`removed`/`left`
- `consecutive_misses` INT (default 0), `total_attended`, `total_missed`
- `removal_reason` (no_show, requested, admin)

`meetings`

- `pod_id` FK, `zoom_meeting_id`, `scheduled_at`, `duration_minutes`
- `status`: `scheduled` ·`completed`/`cancelled`
- `transcript_text` TEXT, `summary` TEXT, `action_items` JSONB,

resources_mentioned JSONB
- followup_sent_at

meeting_attendance

- meeting_id FK, client_id FK (unique together)
- status: expected · attended/absent/excused
- join_time, leave_time, duration_seconds

email_communications

- client_id FK, pod_id FK, agentmail_message_id, agentmail_thread_id
- direction (outbound/inbound), email_type (pod_invitation, bio_for_approval, meeting_reminder, meeting_followup, no_show_warning, removal_notice, etc.)
- response_needed, response_received, response_type (yes/no/maybe)

client_match_scores

- client_a_id FK, client_b_id FK (unique, canonical ordering a < b)
- similarity_score, competition_score, compatibility_score
- Component scores: timezone_score, region_score, revenue_score, stage_score, industry_distance

backfill_queue

- client_id FK, reason (initial_unmatched, declined, removed_from_pod, new_client)
- priority INT, status: waiting · matching · matched/expired

audit_log

- entity_type, entity_id, action, details JSONB, performed_by

admin_users

- email, password_hash, name, role (super_admin/admin/viewer)

system_settings

- Key-value store for configurable parameters (pod_max_size=8, consecutive_miss_limit=2, meeting_cadence=monthly, etc.)

Core Modules

1. Typeform Intake Processor

File: app/services/intake_processor.py

- **Webhook receiver** at POST /webhooks/typeform · Typeform fires on each submission
- Parses the Typeform webhook payload, extracts answers by field ID mapping

- Stores raw response in `clients.intake_form_raw`
- Enqueues Celery task `analyze_intake`

Claude analysis uses `tool_use` for guaranteed structured output:

- Classifies: industry, niche, business model, revenue range, employee range, business stage
- Normalizes: timezone (IANA), region, country, city
- Extracts: primary challenge, goals
- Returns confidence score; flags for admin review if < 0.7

2. Client Matching Engine

File: `app/services/matching_engine.py`

Compatibility formula:

```
compatibility = similarity_score - (competition_penalty *
competition_score)
similarity_score = 0.35 * timezone + 0.20 * region + 0.25 * revenue
+ 0.20 * stage
competition_score:
  same industry AND same niche = 1.0 (HARD BLOCK · never in same pod)
  same industry, different niche = 0.5 (soft penalty)
  different industry = 0.0
```

Timezone scoring: same=1.0, 1h offset=0.8, 2h=0.6, 3h=0.3, 4h+=0.0

Grouping algorithm (greedy graph-based):

1. Precompute all pairwise scores (~500K pairs for 1,000 clients; takes <30s)
2. Sort clients by timezone/region to seed natural clusters
3. For each pod: seed with highest-compatibility pair, then greedily add best-fitting client
4. Hard constraint: no two clients with competition_score=1.0 in same pod
5. Pods with <4 members after grouping · members go to backfill queue

3. Pod Formation Service

File: `app/services/pod_formation.py`

Orchestrates: matching · pod creation · invitation · confirmation tracking · activation. Generates pod names using NATO phonetic alphabet (Pod Alpha, Pod Bravo, etc.).

4. Email Communication Service (AgentMail)

File: `app/services/email_service.py`

- One **system inbox** (`mastermind@yourdomain.com` via AgentMail custom domain)
- Optionally per-pod inboxes for thread isolation
- **Webhook handler** at `POST /webhooks/agentmail` processes inbound replies
- Claude classifies every reply (yes/no/maybe/question/change request) and routes accordingly
- All emails are **Claude-generated per-client** (not static templates) · professional, warm, concise (<200 words), varied language to avoid spam filters

Email types: pod_invitation, invitation_reminder (day 3), pod_confirmation, bio_for_approval, bio_approved, meeting_reminder_24h, meeting_reminder_1h, meeting_followup, no_show_warning, removal_notice, replacement_welcome

5. Zoom Integration Service

File: `app/services/zoom_service.py`

- **Auth:** Server-to-Server OAuth (account_id, client_id, client_secret)
- **Create recurring monthly meeting** per pod (type 8, monthly recurrence, cloud recording enabled, join_before_host=true)
- **Scheduling conflict avoidance:** Before creating a meeting, query existing meetings for that Zoom user at the proposed time; shift by 30-min increments if conflict
- **Post-meeting:** GET `/v2/report/meetings/{id}/participants` for attendance; GET `/v2/meetings/{id}/recordings` for transcript (VTT format)
- Required Zoom scopes: `meeting:write:admin`, `recording:read:admin`, `report:read:admin`

6. Bio Summary Generator

File: `app/services/bio_service.py`

- Claude generates 3-4 sentence professional bio from intake data (60-80 words)
- Excludes: revenue, financials, contact info
- **Approval flow:** generate · email to client · client replies APPROVE or suggests changes · Claude incorporates changes · re-send (max 3 rounds) · auto-approve after 5 days if no response

7. Pod Page Service

File: `app/services/pod_page_service.py` + `app/templates/pod_page.html`

- **URL:** `https://mastermind.yourdomain.com/pods/{access_token}` (64-char hex, 256 bits of entropy)
- Shows: pod name, meeting schedule, member bios (first name + last initial), contact info per member (email only, with consent)
- **Dynamic:** reads from DB on each request · members added/removed reflect immediately
- Clean Tailwind CSS design, no JS framework needed

8. Attendance Monitor & Enforcement

File: `app/services/attendance_service.py`

- Runs 30 min after each meeting ends (Zoom processing delay)
- Matches Zoom participant list against pod members (by email, duration > 5 min = attended)
- Updates `consecutive_misses` counter
- **1st miss:** warning email
- **2nd consecutive miss:** removal · removal notice email · add to backfill queue · trigger replacement search
- Admin can mark absences as "excused" (doesn't count as miss)

9. Backfill Service

File: `app/services/backfill_service.py`

- When a pod has an open slot: query backfill queue · score candidates against existing pod members · enforce non-competition · invite top match
- If no suitable candidate: flag pod for admin, optionally widen timezone tolerance by 1 hour

10. Admin Dashboard

File: `app/routers/admin.py` + `app/templates/admin/`

Pages: | Page | URL | Key Metrics | |------|-----|-------------| | Overview | `/admin/` | Total pods (by status), total clients, meetings this month, response rates, attendance rates | | Clients | `/admin/clients` | Searchable/filterable table, status badges, pod assignment, attendance stats | | Client Detail | `/admin/clients/{id}` | Intake data, Claude analysis, email history, attendance record, bio status | | Pods | `/admin/pods` | All pods with member count, status, next meeting, health score | | Pod Detail | `/admin/pods/{id}` | Members, meeting history with summaries, pod page link, manual actions | | Emails | `/admin/emails` | All sent/received, filterable by type/client/pod, thread viewer | | Meetings | `/admin/meetings` | Calendar view, past meetings with attendance/summaries/action items | | Backfill Queue | `/admin/backfill` | Waiting clients, manual matching controls | | Settings | `/admin/settings` | System configuration, API key status, webhook health |

Tech: HTMX for dynamic updates (auto-refreshing stats, inline editing) · no React/Vue needed for an internal tool.

Automation Workflows

Workflow 1: New Client · Pod Assignment

```
Typeform submission
  · webhook to /webhooks/typeform
  · store raw data, status=intake_received
  · Celery: analyze_intake (Claude tool_use · structured
classification)
  · status=analyzed
  · Daily batch: if ·8 unassigned clients · run matching · form pods
  · Send invitation emails via AgentMail
  · AgentMail webhook receives replies · Claude classifies · update
status
  · When pod has ·4 confirmations · activate pod
  · Generate bios · send for approval · create Zoom meeting · send
confirmation emails
```

Workflow 2: Monthly Meeting Cycle

```
Celery Beat: 24h before meeting · send reminder emails
Celery Beat: 1h before meeting · send short reminder with Zoom link
Meeting happens on Zoom (cloud recorded, auto-transcribed)
30 min after meeting ends · Celery: fetch attendance from Zoom Report
API
  · mark attended/absent · update consecutive_misses
2h after meeting · Celery: fetch recording/transcript
  · Claude: extract summary, action items, resources
  · Send personalized follow-up email to each attendee
  · Send abbreviated summary to absentees
```

Workflow 3: No-Show Handling

```
1st miss · send warning email ("We noticed you missed...")
2nd consecutive miss · remove from pod
  · send professional removal notice
```

```
  · add to backfill queue (can be re-matched if they express
interest)
  · trigger backfill: find replacement from queue
  · invite replacement · bio generation · pod page update
  · notify existing members of new member
```

Workflow 4: Bio Approval

```
Claude generates bio · email to client for review
Client replies APPROVE · publish to pod page · confirm via email
Client suggests changes · Claude regenerates · re-send (max 3 rounds)
No response after 5 days · auto-approve · notify admin
```

Scheduled Tasks (Celery Beat)

| Schedule | Task | Purpose | |----------|------|---------| | Every 5 min | `poll_meeting_status` | Backup check for ended meetings (in case Zoom webhook missed) | | Daily 8:00 AM UTC | `check_matching_batch` | Run matching if ·4 unassigned clients | |Daily 9:00 AM UTC | `send_meeting_reminders_24h` | 24-hour meeting reminders | | Hourly | `send_meeting_reminders_1h` | 1-hour meeting reminders | | Daily 6:00 AM UTC | `check_invitation_expirations` | Handle expired invitations (7 days no response) | | Daily 6:00 AM UTC | `check_bio_approval_timeouts` | Auto-approve bios past 5-day timeout | | Daily 7:00 AM UTC | `process_backfill_queue` | Fill open pod slots | | Every 30 min | `sync_zoom_recordings` | Fetch transcripts for completed meetings | | Weekly (Mon 2 AM UTC) | `generate_weekly_report` | Admin report: pod health, attendance trends |

Project Structure

```
mastermind-pods/
··· docker-compose.yml              # PostgreSQL, Redis, app,
worker, beat
··· Dockerfile                       # FastAPI app
··· Dockerfile.worker                # Celery worker
··· pyproject.toml                   # Dependencies
··· alembic.ini
··· .env.example
··· alembic/versions/                # DB migrations
··· app/
·   ··· main.py                      # FastAPI app factory
·   ··· config.py                    # pydantic-settings
·   ··· db/session.py                # AsyncSession + engine
·   ··· models/                      # SQLAlchemy ORM models (1 file
per table)
·   ··· schemas/                     # Pydantic request/response
schemas
·   ··· routers/
·   ·   ··· clients.py               # /api/v1/clients/*
·   ·   ··· pods.py                  # /api/v1/pods/*
·   ·   ··· meetings.py              # /api/v1/meetings/*
·   ·   ··· matching.py              # /api/v1/matching/*
·   ·   ··· stats.py                 # /api/v1/stats/*
```

```
·   ·   ··· webhooks.py                # /webhooks/typeform,
/webhooks/agentmail, /webhooks/zoom
·   ·   ··· public.py                  # /pods/{access_token}
·   ·   ··· admin.py                   # /admin/* (dashboard)
·   ··· services/                      # Business logic (1 file per
module above)
·   ··· tasks/
·   ·   ··· celery_app.py              # Celery config + Beat schedule
·   ·   ··· intake_tasks.py
·   ·   ··· matching_tasks.py
·   ·   ··· email_tasks.py
·   ·   ··· zoom_tasks.py
·   ·   ··· bio_tasks.py
·   ·   ··· attendance_tasks.py
·   ·   ··· backfill_tasks.py
·   ·   ··· scheduled_tasks.py
·   ··· ai/
·   ·   ··· client.py                  # Anthropic client singleton + rate
limiter
·   ·   ··· prompts.py                 # All prompt templates
·   ·   ··· tools.py                   # Claude tool_use definitions
·   ·   ··· parsers.py                 # Response validation
·   ··· integrations/
·   ·   ··· agentmail_client.py        # AgentMail SDK wrapper
·   ·   ··· zoom_client.py             # Zoom API client (httpx)
·   ·   ··· typeform_client.py         # Typeform API client (httpx)
·   ··· templates/
·   ·   ··· pod_page.html              # Public pod page
·   ·   ··· admin/                     # Dashboard templates
·   ·   ··· emails/                    # Email HTML base layouts
·   ··· static/                        # Tailwind CSS, HTMX, logo
··· tests/                             # pytest + pytest-asyncio
··· scripts/                           # seed_data.py,
create_admin.py, import_csv.py
```

Implementation Sequence

Phase	Scope	Files
1. Foundation	Project scaffold, Docker Compose, DB models, Alembic migrations, FastAPI shell, Celery+Redis	`docker-compose.yml`, `app/main.py`, `app/models/*`, `alembic/`
2. AI + **Intake**	Claude service wrapper, Typeform webhook receiver, intake analysis pipeline	`app/ai/*`, `app/services/intake_processor.py`, `app/integrations/typeform_client.py`
3. Matching	Pairwise scoring, grouping algorithm, pod formation	`app/services/matching_engine.py`, `app/services/pod_formation.py`
4. Email	AgentMail integration, invitation flow, reply processing	`app/integrations/agentmail_client.py`, `app/services/email_service.py`, `app/routers/webhooks.py`
5. Zoom	Meeting creation, scheduling, conflict avoidance	`app/integrations/zoom_client.py`, `app/services/zoom_service.py`
6. Bio + **Pages**	Bio generation, approval flow, pod page rendering	`app/services/bio_service.py`, `app/templates/pod_page.html`
7. Post-Meeting	Attendance tracking, transcript analysis, follow-up emails, no-show enforcement	`app/services/attendance_service.py`, `app/tasks/meeting_tasks.py`
8. Backfill	Replacement matching, re-invitation	`app/services/backfill_service.py`
9. Admin	Dashboard pages, auth, stats API, settings	`app/routers/admin.py`, `app/templates/admin/*`
10. Harden	E2E tests, error handling, load

testing, deliverability testing, security review | `tests/*` |

Suggestions for Improvement

1. **NPS / Pod Health Surveys**: After every 3rd meeting, send a 1-question survey ("On a scale of 1-10, how valuable is your mastermind pod?"). Track pod health scores over time. Low-scoring pods get flagged for admin intervention or reshuffling.
2. **Smart Re-Matching (Quarterly)**: Every quarter, offer members the option to rotate into a new pod. Some members benefit from fresh perspectives. The system could automatically propose "pod refreshes" based on tenure and satisfaction scores.
3. **Pre-Meeting Agenda Bot**: 48h before each meeting, email each member asking: "What's your #1 win and #1 challenge to discuss?" Compile responses and share with the pod 24h before so members come prepared. This dramatically improves meeting quality.
4. **Accountability Tracking**: After each meeting's action items are extracted, the system follows up individually 2 weeks later: "You committed to X. How did it go?" Track completion rates per member and per pod.
5. **Warm Introductions**: When a new member joins an existing pod, send each existing member a 1:1 email with the new member's bio and a suggested "get to know each other" prompt. This reduces the cold-start problem.
6. **Meeting Quality Scoring**: Use Claude to score each meeting transcript on dimensions like "equal participation" (did everyone talk?), "action-oriented" (were concrete next steps defined?), "depth" (surface-level vs. deep discussion). Flag low-quality meetings for facilitator tips.
7. **Resource Library**: Aggregate all resources/tools/books mentioned across ALL pods into a searchable library. Members can browse what others have recommended. Tag by topic and track which resources are mentioned most frequently.
8. **Slack/WhatsApp Integration**: Give each pod an optional async communication channel between meetings. The system could create a private Slack channel or WhatsApp group automatically. The AI agent monitors the channel and includes key discussions in the next meeting follow-up.
9. **Custom Domain for Emails**: Use AgentMail's custom domain feature (paid plan) to send from `mastermind@danmartell.com` rather than `@agentmail.to`. Critical for deliverability and trust.
10. **Gradual Onboarding**: Don't invite all 1,000 clients at once. Roll out in waves of 50-100 per week to test the system, warm up email sending reputation, and iterate on the matching algorithm based on early feedback.

Verification Plan

1. **Unit tests**: Each service module (matching engine, attendance, bio generation) tested with pytest
2. **Integration tests**: Full workflow tests (intake · analysis · matching · invitation · confirmation · meeting · follow-up) using test doubles for external APIs
3. **Staging environment**: Deploy to a staging VPS with test Typeform, AgentMail sandbox, and Zoom sandbox accounts
4. **Pilot cohort**: Run with 24 real clients (3 pods of 8) for one full meeting cycle before scaling
5. **Load test**: Simulate 1,000 clients with synthetic data to verify matching performance and email throughput
6. **Email deliverability**: Send test emails to Gmail, Outlook, Yahoo test accounts to verify inbox placement
7. **Admin dashboard**: Manual walkthrough of all pages with the pilot data